

Naive Bayes with Numerical Data: Gaussian Naive Bayes

Contents

1. Overview	2
2. The problem	2
3. The assumption: normal distributions	3
4. The numbers	3
5. When the data is not normal	4
6. Summary	6
7. Key terms	6

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Classification problems ([Note 3](#)); Independent and mutually exclusive events ([Note 84](#)); Bayes' theorem ([Note 86](#)).

1. Overview

Key point: Numerical inputs rarely repeat exactly, so counting fails. Gaussian Naive Bayes assumes each input follows a normal distribution within each class, and uses the normal curve's height (density) in place of a counted probability.

So far every input was categorical, so probabilities came from counting. This Note handles **numerical** inputs such as height or weight, and introduces **Gaussian Naive Bayes**, the most common variant for them.

2. The problem

Key point: No man in the data is exactly 185 cm tall, so $P(\text{height} = 185 \mid \text{male})$ by counting is 0.

The data has 8 people with height (cm), weight (pounds) and gender:

Height (cm)	Weight (lb)	Gender
182.9	180	male
180.4	190	male
170.1	170	male
180.4	165	male
152.4	100	female
167.6	150	female
165.2	130	female
175.3	150	female

A new person is 185 cm tall and weighs 170 pounds. Male or female?

The Naive Bayes recipe (the maths Note) needs

$$P(\text{male}) \times P(\text{height} = 185 \mid \text{male}) \times P(\text{weight} = 170 \mid \text{male})$$

and the same for female. The prior is easy: 4 of 8 are male, so $P(\text{male}) = 1/2$. But no man in the data is exactly 185 cm, so counting gives $P(\text{height} = 185 \mid \text{male}) = 0$, and the whole

score is 0. With a continuous measurement, almost every new value is one never seen before, so counting cannot work.

3. The assumption: normal distributions

Key point: Assume each column is normally distributed within each class. Estimate the mean and standard deviation, then read off the curve's height at the new value.

Gaussian Naive Bayes assumes that, within each class, each numerical input follows a **normal** (Gaussian) distribution, the bell curve of the earlier statistics Notes.

For each class and each column:

1. compute the **mean** μ and **standard deviation** σ of that column for that class's rows;
2. for a new value x , compute the normal density

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

3. use $f(x)$ in place of $P(x \mid \text{class})$ in the Naive Bayes product.

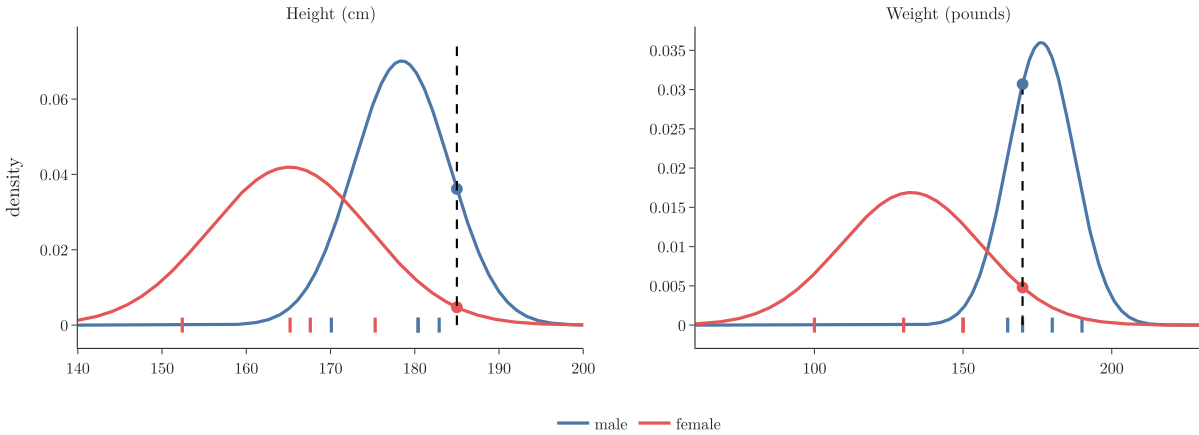


Figure 1: A normal curve per class for each column; the dots mark the new person's values

Figure 1 shows the four fitted curves. The new person's height (185 cm) sits on the right flank of the male curve but far out in the tail of the female curve; the same holds for the weight (170 pounds).

4. The numbers

Key point: Male score 5.5×10^{-4} against female 1.1×10^{-5} : predict male, with about 98% probability.

Class	Column	Mean	Std. deviation	Density at the new value
male	height	178.45	5.69	0.03615
male	weight	176.25	11.09	0.03070
female	height	165.12	9.51	0.00473
female	weight	132.50	23.63	0.00479

$$\text{male: } 0.5 \times 0.03615 \times 0.03070 = 5.5 \times 10^{-4}$$

$$\text{female: } 0.5 \times 0.00473 \times 0.00479 = 1.1 \times 10^{-5}$$

The male score is about 49 times larger, so the prediction is **male**. As probabilities: 98% male, 2% female.

Extra: A density is not a probability: for a continuous variable, the probability of exactly 185.000... cm is 0, and densities can even be larger than 1. But the density measures how likely values **near** 185 are, and the same small interval would multiply every class's density equally. So comparing densities across classes gives the same decision as comparing probabilities.

Python: scikit-learn's GaussianNB.

```
from sklearn.naive_bayes import GaussianNB
```

```
nb = GaussianNB().fit(df[["height_cm", "weight_lb"]], df["gender"])
nb.theta_          # class means
nb.var_            # class variances
nb.predict_proba(new_person) # about 99% male
```

GaussianNB estimates the variance by dividing by n rather than $n - 1$ (and adds a tiny amount for numerical safety), so its numbers differ slightly from the table: 99.2% male instead of 98.0%. The prediction is the same.

5. When the data is not normal

Key point: The normal assumption can be wrong. Other variants assume other distributions: multinomial for counts, Bernoulli for yes/no, categorical for categories.

The normal assumption is a choice, and it can be poor, for example for a skewed column like income. Naive Bayes can use any distribution that suits the data, giving different variants:

Variant	Assumed distribution	Typical inputs	scikit-learn
Gaussian	normal	continuous measurements	GaussianNB

Variant	Assumed distribution	Typical inputs	scikit-learn
Multinomial	multinomial	counts, such as word counts in a document	MultinomialNB
Bernoulli	Bernoulli (yes/no)	binary features, such as “word present or not”	BernoulliNB
Categorical	categorical	categories, as in the Play Tennis Note	CategoricalNB

The approach is to look at each column’s distribution and pick the variant whose assumption fits best. A strongly skewed numerical column can also be transformed first (the power transformer Note) so that it looks more normal.

6. Summary

- Numerical values rarely repeat, so $P(x \mid \text{class})$ cannot be counted.
- Gaussian Naive Bayes fits a normal curve (mean, standard deviation) per class and column, and uses the density at x .
- New person 185 cm, 170 lb: male score 5.5×10^{-4} , female 1.1×10^{-5} : male (98%).
- Other variants (multinomial, Bernoulli, categorical) suit other kinds of inputs.

7. Key terms

Term	Meaning
Gaussian Naive Bayes	Naive Bayes that models each numerical input as normally distributed within each class
Probability density	The height of a continuous distribution's curve; compares how likely nearby values are
GaussianNB	scikit-learn's Gaussian Naive Bayes
MultinomialNB	Naive Bayes for count data, such as word counts
BernoulliNB	Naive Bayes for binary (yes/no) inputs